

API операционных систем

Межпроцессное взаимодействие

Подготовил
Старший преподаватель
МИЭМ НИУ ВШЭ
Морозов Владимир Игоревич

Общая информация

Основные положения

- Ранее для передачи данных между процессами мы использовали простые файлы
- Это удобно, т.к. позволяет отслеживать передаваемые данные
- Тем не менее, такой способ взаимодействия между процессами слабо распространён, т.к. требует обращений к диску, что может значительно замедлить работу приложений в случае активного обмена данными
- Чтобы избежать потерь в производительности, в разное время были созданы различные средства межпроцессного взаимодействия (IPC)
- Часть из них будет рассмотрена в рамках данного занятия

Linux

Основные положения

- В Linux предоставляется широкий спектр средств IPC, предназначенных для различных целей
- Некоторые из этих средств были разработаны в качестве замены аналогичным, но более старым, инструментам
- Таким образом, часть средств IPC в Linux практически дублирует друг друга за тем исключением, что более новые предоставляют более удобный и менее подверженный возникновению ошибок функционал
- Мы рассмотрим два распространённых средства: каналы (pipes) и разделяемую память стандарта POSIX (Posix shared memory)

Каналы

Каналы

- Каналы (pipes) – наиболее старое и примитивное средство межпроцессного взаимодействия
- Каждый канал представляет собой пару дескрипторов, один из которых доступен на чтение, другой – на запись
- Писать в канал и читать из него можно так же, как и в любой другой файл или консоль
- Наименее тривиальной процедурой является передача канала другому процессу

Функции для работы с каналом

- Создание канала производится посредством вызова функции `pipe`, принимающей единственный аргумент:
 - `int pipefd[2]` – массив из двух чисел типа `int` (выделенный в динамической памяти или стеке); в первый элемент этого массива будет записан дескриптор для чтения, во второй – дескриптор для записи
- Запись в канал производится обычной функцией `write`, которой, вместо дескриптора файла, передаётся дескриптор записываемого “конца” канала
- Чтение производится функцией `read`, которой передаётся считываемый “конец” канала
- Закрытие дескриптора канала, как и любого файла, производится функцией `close`

Пример кода

- На следующем слайде представлен простейший пример кода, не выполняющих каких-либо полезных действий
- Процесс создаёт канал, сам записывает туда строку `"Hello\n"` после чего сам же читает её и выводит в консоль
- Обработка ошибок в примере опущена для сокращения кода
- Чтобы представленный код работал, необходимо подключить заголовочный файл `unistd.h`

```
int fds[2];  
if (pipe(fds) < 0) {  
    // Обработка ошибки  
}  
write(fds[1], "Hello\n", 7);  
  
char buf[7];  
read(fds[0], buf, 7);  
  
write(STDOUT_FILENO, buf, 7);
```

Блокировка при чтении

- Чтение из канала по принципу работы больше похоже на чтение из консоли, чем из файла: в случае, если данных в канале недостаточно, вызов будет заблокирован до появления запрошенного количества
- Блокировка также может закончиться, если записываемый дескриптор канала будет закрыт функцией `close`
- Чтобы не сталкиваться с блокировками работы программы, существуют два распространённых подхода:
 - Передавать данные сообщениями фиксированной длины
 - В начале сообщения передавать длину оставшейся части в виде числа фиксированной байтовой длины
- Второй подход, как правило, является предпочтительным

Передача дескрипторов

- Предыдущий пример не выполняет никакой полезной работы, т.к. оба дескриптора канала используются одним и тем же процессом
- Т.к. канал является неименованным, его нельзя открыть из другого процесса по какому-либо уникальному идентификатору
- Чтобы передать тот же канал другому процессу, используется свойство **наследования дескрипторов**
- Оно состоит в том, что, когда процесс выполняет `fork()`, дочерний процесс получает возможность использовать все дескрипторы, которые были открыты родительским процессом
- Наибольшую сложность вызывает передача такого дескриптора новому исполняемому файлу при вызове `exec()`

Передача дескрипторов

- Существует ряд способов передать дескрипторы канала новому процессу, выполнившему `exec()`
- Наиболее распространённым из них является замена дескрипторов стандартного ввода или вывода (или и того, и другого) нового процесса на дескрипторы канала
- Для этого используется функция дублирования дескриптора

Дублирование дескриптора

- Для дублирования дескриптора используется функция `dup2`, принимающая следующие аргументы:
 - `int oldfd` – номер старого дескриптора файла, консоли, канала и т.п.
 - `int newfd` – новый дескриптор, который после вызова будет ссылаться на тот же файл, консоль канал и т.п.
- Функция возвращает новый дескриптор в случае успешного выполнения и `-1` в случае ошибки
- Код ошибки, как и всегда, записывается в `errno`
- Если `newfd` представляет уже открытый дескриптор, он предварительно автоматически закрывается с помощью `close`

Передача дескриптора после `exec()`

Передача дескриптора новому процессу, вызвавшему `exec()`, осуществляется по следующему алгоритму:

1. Родительский процесс вызывает `fork()`
2. Родительский процесс (получивший `>0`) закрывает, например, читаемый дескриптор канала с помощью `close`
3. Дочерний процесс (получивший `0`) дублирует, например, читаемый дескриптор канала в дескриптор `STDIN_FILENO`, что приводит к закрытию стандартного ввода для дочернего процесса
4. Дочерний процесс выполняет `exec`
5. Теперь любая операция чтения из `STDIN_FILENO` в дочернем процессе будет читать из канала

Передача дескриптора после `exec()`

- После выполнения шагов представленного алгоритма родительский процесс может писать в записываемый дескриптор канала, а дочерний процесс может читать эти данные из `STDIN_FILENO`
- При этом стандартный ввод становится недоступен для дочернего процесса
- На следующих слайдах представлен код для родительского и дочернего процессов, демонстрирующий реализацию представленного алгоритма


```
int fd_pair[2];
pipe(fd_pair);
int parent_write_pipe = fd_pair[0];
int child_read_pipe = fd_pair[1];

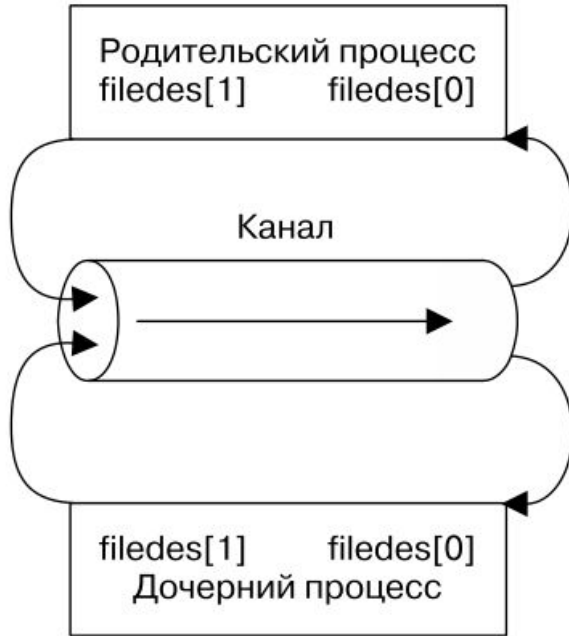
switch(fork()) {
    case -1:
        // Обработка ошибки ...
    case 0:
        dup2(child_read_pipe, STDIN_FILENO);
        close(child_read_pipe);
        close(parent_write_pipe);
        // Выполнение exec() ...
    default:
        close(child_read_pipe);
}

write(parent_write_pipe, "Hello\n", 7);
```

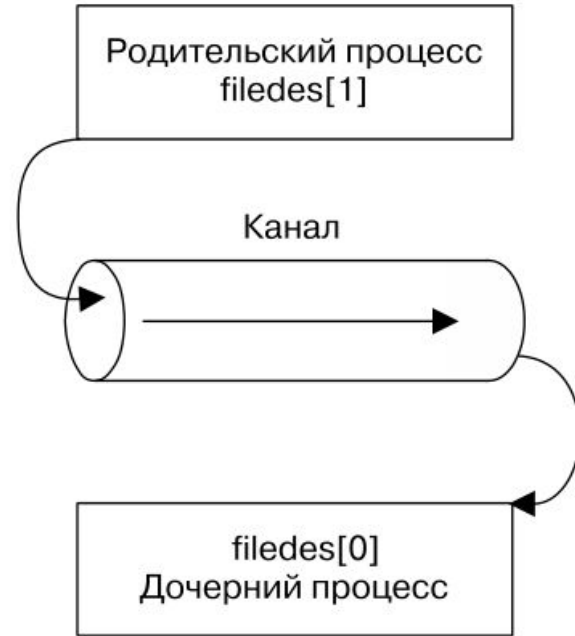
Пример кода

- Дочерний процесс в предыдущем примере может просто выполнить `read` из дескриптора `STDIN_FILENO`, чтобы прочитать данные, переданные родительским процессом
- Процесс может открывать сколько угодно каналов, соответственно, можно открыть дополнительный канал для обратной передачи данных от дочернего процесса родительскому и дублировать его в `STDOUT_FILENO` дочернего процесса

Визуализация из книги



А. После `fork()`



Б. После закрытия неиспользуемых дескрипторов

Необходимость закрытия дескрипторов

- Чтобы взаимодействие по каналам происходило правильно и без ошибок, необходимо **своевременно закрывать все неиспользуемые дескрипторы** с помощью `close`
- Например, если у процесса останется открытым записываемый дескриптор для канала, из которого он читает, он никогда не сможет получить оповещение о конце записи (`read` никогда не вернёт 0), которое отправляется, когда все записываемые дескрипторы закрыты

Недостатки каналов

- Сложность передачи дескриптора в другой процесс
- Могут взаимодействовать только родственные процессы (за редкими исключениями)
- Нет границ сообщения, только байтовый поток
- Если записать в канал, для которого не открыт читаемый дескриптор с другой стороны, система пошлёт сигнал, аварийно завершающий процесс

Преимущества каналов

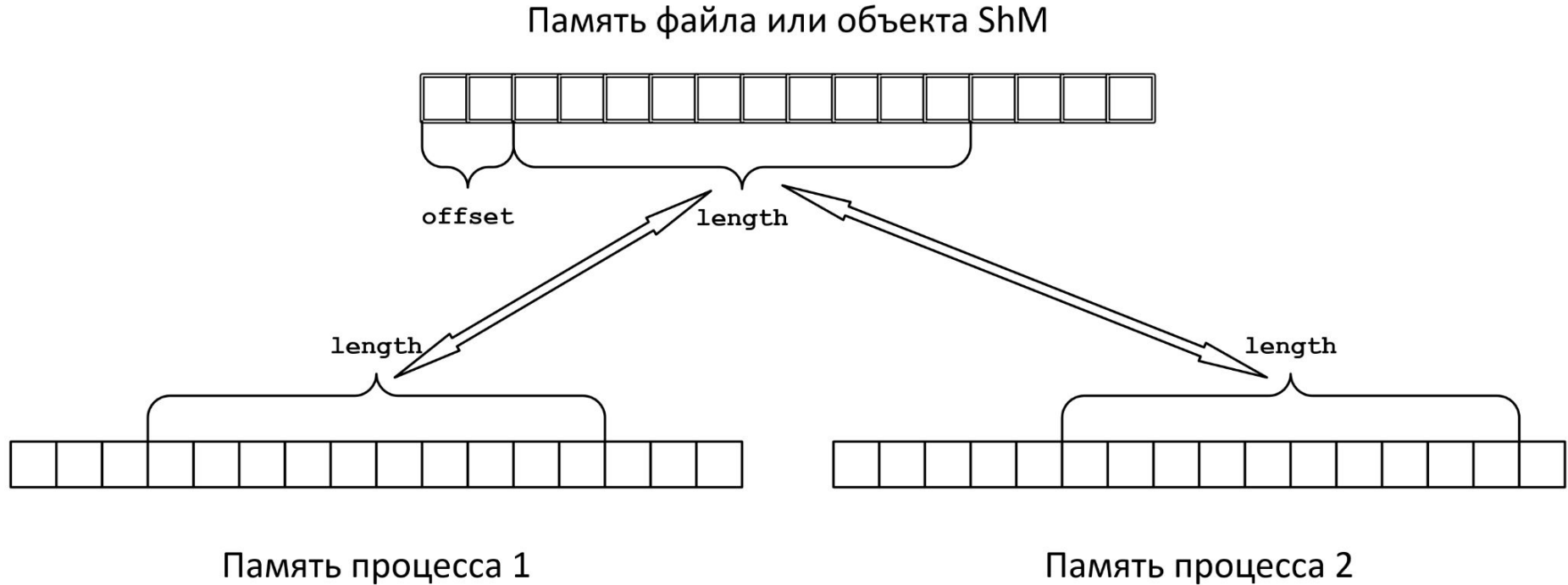
- Можно использовать даже с теми процессами, программы которых не разрабатывались для работы с каналами (особенно с теми, которые разработаны не вами); **так работает конструкция | в bash.**
- Нет необходимости синхронизировать время чтения и записи, т.к. чтение блокируется до поступления данных
- Простой принцип работы, всего 1 новый системный вызов

Разделяемая память

Разделяемая память

- Разделяемая память – один из подвидов более масштабного средства IPC – файловых отображений
- Такие отображения позволяют загрузить часть файла или весь файл в ОЗУ и работать с ним, как с обычной динамически выделенной памятью (через указатели)
- При этом все манипуляции с памятью транслируются в файл на диске
- Разделяемая память – это отображение, не ассоциированное с конкретным файлом, но при этом доступное другим процессам по уникальному имени

Файловое отображение



Работа с разделяемой памятью

Работа с разделяемой памятью включает следующие шаги:

- Создание объекта разделяемой памяти (один процесс) **или** открытие такого объекта (много процессов)
- Задание размера разделяемой памяти (один процесс)
- Отображение объекта разделяемой памяти на адресное пространство процесса (много процессов)
- Работа с отображённой памятью (чтение и запись по указателю, много процессов)
- Удаление отображения при необходимости (много процессов)
- Удаление объекта разделяемой памяти при необходимости (один процесс), объект удалится сам при перезагрузке системы

Создание объекта ShM

- Для создания объекта ShM используется функция `shm_open`, принимающая следующие аргументы:
 - `const char *name` – имя объекта разделяемой памяти, должно начинаться с /
 - `int oflag` – флаги открытия: `O_RDONLY`, `O_RDWR`, `O_CREAT`, `O_EXCL`, `O_TRUNC`
 - `mode_t mode` – права для вновь создаваемого объекта ShM (как для файлов)
- Функция возвращает дескриптор, аналогичный файловому, который можно использовать для отображения объекта разделяемой памяти на адресное пространство процесса
- В случае ошибки функция вернёт `-1`

Задание размера ShM

- Для задания размера ShM (после создания размера объект имеет размер 0) используется функция `ftruncate`, принимающая следующие аргументы:
 - `int fildes` – дескриптор объекта разделяемой памяти
 - `off_t length` – необходимый размер разделяемой памяти
- В случае успешного выполнения функция возвращает 0, в случае ошибки `-1`

Отображение объекта ShM

- Для отображения объекта ShM на адресное пространство процесса используется функция `mmap`, принимающая следующие аргументы:
 - `void * addr` – адрес, по которому необходимо расположить отображённую память (будем устанавливать в `NULL` для автоматического расположения)
 - `size_t length` – длина отображаемого участка (размер отображения) в байтах
 - `int prot` – флаги защиты памяти:
 - `PROT_EXEC` – отображённый участок памяти можно выполнить
 - `PROT_READ` – участок памяти можно прочитать
 - `PROT_WRITE` – в участок памяти можно записать
 - `int flags` – флаг видимости другим процессам, в нашем случае – `MAP_SHARED`
 - `int fd` – дескриптор объекта ShM (в нашем случае созданного в `shm_open`)
 - `off_t offset` – сдвиг отображения относительно начала объекта ShM (ставим 0)
- Функция возвращает указатель `void*` на отображённый участок памяти (как `malloc`) в случае успеха и `MAP_FAILED` в случае ошибки

Пример кода

- На следующем слайде представлен пример кода, в котором производятся описанные на предыдущих слайдах операции, и в полученное отображение записывается строка `Hello`
- Далее (на другом слайде) представлен код дочернего процесса, получающего эту строку и выводящего её на экран
- Обработка ошибок опущена для сокращения примера

Родительский процесс

```
int fd = -1;
fd = shm_open("/pr7", O_CREAT | O_RDWR, S_IRUSR | S_IWUSR);

ftruncate(fd, 7);

char * memory = MAP_FAILED;
memory = mmap(NULL, 7, PROT_READ | PROT_WRITE, MAP_SHARED,
fd, 0);

memcpy(memory, "Hello\n", 7);

// Запуск дочернего процесса...
```

Дочерний процесс

```
int fd = -1;

fd = shm_open("/pr7", O_RDWR, S_IRUSR | S_IWUSR);

char * memory = MAP_FAILED;

memory = mmap(NULL, 7, PROT_READ | PROT_WRITE,
MAP_SHARED, fd, 0);

write(STDOUT_FILENO, memory, 7);
```


Удаление отображения

- Чтобы удалить отображение для данного процесса, используется функция `munmap`, принимающая следующие аргументы:
 - `void * addr` – адрес удаляемого участка
 - `size_t length` – размер удаляемого участка
- Удалённый участок больше недоступен для процесса
- Запись и чтение по адресам из такого участка приводит к неопределённому поведению
- Функция `mmap` возвращает 0 в случае успеха и -1 в случае ошибки
- После удаления записанные в отображение данные всё ещё сохраняются в объекте разделяемой памяти

Удаление объекта ShM

- Чтобы удалить объект разделяемой памяти, используется функция `shm_unlink`, принимающая имя удаляемого объекта
- Функция возвращает 0 при успешном завершении и -1 в случае ошибки
- Удаление объекта ShM делает его недоступным для открытия другими процессами, но при этом **никак не влияет на уже созданные отображения**
- Т.е. после удаления объекта ShM процессы всё ещё могут обмениваться данными через свои отображения
- Объект ShM будет окончательно удалён, когда все использующие его процессы завершатся или выполнят `munmap`
- Заккрытие дескриптора ShM с помощью `close` не оказывает никакого влияния, если выполнено после `ftruncate` и `mmap`

Синхронизация

Необходимость синхронизации

- В отличие от каналов, где чтение автоматически блокируется до получения достаточного количества данных, что позволяет не заботиться о синхронизации, обращения к разделяемой памяти должны быть синхронизированы
- Это обосновано также тем, что при взаимодействии ряда процессов через каналы, как правило, используется по паре каналов на каждую пару процессов, а объекты разделяемой памяти часто создаются в единственном экземпляре для всех процессов

Средства синхронизации ShM

- Самым простым способом синхронизации действий различных процессов являются мьютексы POSIX
- Т.к. изначально они используются для синхронизации потоков, их нельзя открыть из другого процесса по имени (как в Windows)
- Чтобы использовать мьютекс из разных процессов, необходимо расположить его в участке разделяемой памяти (можно создать отдельный объект ShM для мьютекса)
- Необходимо обращать внимание, что такой мьютекс нельзя инициализировать с помощью `PTHREAD_MUTEX_INITIALIZER`

Размещение мьютекса в ShM

Чтобы разместить мьютекс в разделяемой памяти, необходимо:

1. Создать объект разделяемой памяти
2. Задать его длину равной размеру типа `pthread_mutex_t`
3. Отобразить разделяемую память на адресное пространство процесса
4. Записать новый мьютекс в разделяемую память с помощью `pthread_mutex_init`
5. В других процессах открывать объект ShM, отображать его в процесс и использовать полученный указатель в функциях открытия и закрытия мьютекса
6. По окончании работы мьютекс можно удалить из отображения, используя `pthread_mutex_destroy`

Создание мьютекса в памяти

- Чтобы создать мьютекс в динамически выделенном участке памяти, используется функция `pthread_mutex_init`, принимающая следующие аргументы:
 - `pthread_mutex_t *mutex` – указатель на участок памяти, в котором необходимо разместить мьютекс
 - `const pthread_mutexattr_t *attr` – указатель на структуру с атрибутами мьютекса
- Функция возвращает 0 при успешном завершении и код ошибки в случае ошибки
- Структура `pthread_mutexattr_t` должна быть предварительно проинициализирована функцией `pthread_mutexattr_init` (см. пример кода далее)

Пример кода

- На следующем слайде представлен пример кода родительского процесса, в котором мьютекс создаётся и размещается в разделяемой памяти
- Для задания атрибутов мьютекса, требуемых функцией `pthread_mutex_init`, используется функция `pthread_mutexattr_init`
- Дочерний процесс, в свою очередь, должен только отображать соответствующую память и использовать `lock` и `unlock` для синхронизации


```
int mfd = -1;
mfd = shm_open("/pr7mutex", O_CREAT | O_RDWR, S_IRUSR | S_IWUSR);

ftruncate(mfd, sizeof(pthread_mutex_t);

pthread_mutex_t * mutex_memory = MAP_FAILED;
mutex_memory = mmap(NULL, sizeof(pthread_mutex_t), PROT_READ | PROT_WRITE,
MAP_SHARED, mfd, 0);

pthread_mutexattr_t mta;
pthread_mutexattr_init(&mta);
pthread_mutex_init(mutex_memory, &mta);

// Синхронизация с помощью lock и unlock...

pthread_mutex_destroy(mutex_memory);
```

Преимущества и недостатки разделяемой памяти

- + Можно работать как с обычной динамически выделенной (например, через `malloc`) памятью
- + Доступна всем процессам по уникальному имени
- Необходима дополнительная синхронизация
- Ограниченный размер (хотя можно увеличивать)

Windows

Основные положения

- В Windows также предоставляется широкий спектр средств IPC, предназначенных для различных целей
- Однако в данной ОС существует по одному средству каждого из видов
- Дублирование, как в Linux, отсутствует
- В рамках данного занятия для Windows будут также рассмотрены каналы и разделяемая память
- Логически эти средства IPC организованы так же, как и в Linux

Каналы

Функции для работы с каналом

- Создание канала производится посредством вызова функции **CreatePipe**, принимающей следующие аргументы:
 - **PHANDLE hReadPipe** – указатель на заранее объявленный дескриптор читаемого конца
 - **PHANDLE hWritePipe** – указатель на заранее объявленный дескриптор записываемого конца
 - **LPSECURITY_ATTRIBUTES lpPipeAttributes** – указатель на структуру с атрибутами безопасности (обсуждается далее)
 - **DWORD nSize** – размер буфера канала (можно установить 0 и ОС выберет сама)
- Чтение и запись в канал так же производятся функциями **ReadFile** и **WriteFile**
- Заккрытие неиспользуемого дескриптора, очевидно, происходит с помощью функции **CloseHandle**
- Функция возвращает **BOOL** для сообщения о наличии ошибки

Атрибуты безопасности

- В Linux все дескрипторы родительского процесса наследуются дочерними по умолчанию
- В Windows необходимость наследования необходимо указать отдельно
- Делается это с помощью задания соответствующего поля структуры `SECURITY_ATTRIBUTES`
- Инициализация данной структуры достаточно объёмна, поэтому предлагается вынести соответствующий код в функцию, как показано на следующем слайде
- Функция принимает по указателю объявленные заранее структуры `SECURITY_ATTRIBUTES` и `SECURITY_DESCRIPTOR` и инициализирует их

```
VOID InitializeSecurityAttr(LPSECURITY_ATTRIBUTES attr,  
SECURITY_DESCRIPTOR * sd)  
{  
    attr->nLength = sizeof(SECURITY_ATTRIBUTES);  
    attr->bInheritHandle = TRUE; // Включаем наследование  
    дескрипторов  
    InitializeSecurityDescriptor(sd,  
SECURITY_DESCRIPTOR_REVISION);  
    attr->lpSecurityDescriptor = sd;  
}
```


Замена дескрипторов

- После создания канала необходимо заменить дескрипторы стандартного ввода и вывода родительского процесса на дескрипторы ввода и вывода канала
- При этом необходимо предварительно сохранить изначальные значения стандартных дескрипторов, чтобы вернуть возможность консольного ввода-вывода после запуска дочернего процесса
- Задание стандартного дескриптора производится функцией `SetStdHandle`, принимающей флаг потока (`STD_INPUT_HANDLE`, `STD_OUTPUT_HANDLE` или `STD_ERROR_HANDLE`) и дескриптор, который ему необходимо назначить
- Функция возвращает логическое значения для оповещения об ошибке

Пример кода

- В коде на следующем слайде показано, как можно использовать канал для передачи данных от родительского процесса дочернему
- После выполнения представленного кода родительский процесс может писать в записываемый конец канала данные, а дочерний процесс сможет считать их из дескриптора стандартного ввода (возвращённого `GetStdHandle(STD_INPUT_HANDLE)`)
- Обратите внимание, что при запуске процесса аргументу `bInheritHandles` функции `CreateProcess` необходимо передать `TRUE`
- Обработка ошибок в примере опущена для сокращения кода
- Допускается создание нескольких каналов, например, по одному для передачи данных от родительского процесса дочернему и наоборот

```
HANDLE parent_write_pipe, child_read_pipe;  
SECURITY_ATTRIBUTES attr;  
SECURITY_DESCRIPTOR sd;  
InitializeSecurityAttr(&attr, &sd); // Используем собственную функцию  
  
CreatePipe(&child_read_pipe, &parent_write_pipe, &attr, 0);  
  
HANDLE StdIn = GetStdHandle(STD_INPUT_HANDLE);  
  
SetStdHandle(STD_INPUT_HANDLE, child_read_pipe); // Заменяем дескриптор  
  
// Запуск дочернего процесса: CreateProcess(..., TRUE, ...)  
  
CloseHandle(child_read_pipe);  
  
SetStdHandle(STD_INPUT_HANDLE, StdIn); // Возвращаем консольный ввод
```

Разделяемая память

Разделяемая память

- Разделяемая память в Windows также является подвидом файловых отображений
- Работа с анонимным отображением ведётся примерно по тому же алгоритму, что и в Linux:
 - Создание объекта ShM с помощью `CreateFileMapping` (один процесс) или открытие существующего объекта с помощью `OpenFileMapping` (много процессов)
 - Отображение ShM на адресное пространство процесса с помощью `MapViewOfFile` (много процессов)
 - Работа с отображённой памятью (много процессов)
 - Заккрытие отображение при необходимости с помощью `UnmapViewOfFile` (много процессов)
 - Заккрытие объекта ShM при необходимости с помощью `CloseHandle` (много процессов)
- Объект ShM будет удалён, когда будет закрыт всеми процессами

Создание объекта разделяемой памяти

- Объект разделяемой памяти создаётся с помощью функции создания файловых отображений **CreateFileMapping**, принимающей:
 - **HANDLE hFile** – дескриптор отображаемого файла (будем передавать **INVALID_HANDLE_VALUE** для создания анонимного отображения – ShM)
 - **LPSECURITY_ATTRIBUTES lpFileMappingAttributes** – структура с атрибутами безопасности (будет устанавливать в **NULL**)
 - **DWORD flProtect** – флаги защиты памяти, показывающие, что можно делать с отображённой памятью, в нашем случае – **PAGE_READWRITE**
 - **DWORD dwMaximumSizeHigh** – старшая часть максимального размера (ставим 0)
 - **DWORD dwMaximumSizeLow** – младшая часть максимального размера (ставим нужный размер памяти)
 - **LPCSTR lpName** – имя создаваемого объекта (любые символы, допустимые для файла)
- Функция возвращает дескриптор ShM в случае успешного выполнения и **NULL** в случае ошибки

Открытие объекта разделяемой памяти

- Существующий объект разделяемой памяти можно открыть с помощью функции `OpenFileMapping`, принимающей:
 - `DWORD dwDesiredAccess` – флаг доступа (должен быть такой же, как в последующем `MapViewOfFile`, например, `FILE_MAP_ALL_ACCESS`)
 - `BOOL bInheritHandle` – логический параметр, определяющий, можно ли наследовать возвращаемый дескриптор
 - `LPCSTR lpName` – строка с именем существующего объекта ShM
- Функция возвращает дескриптор ShM в случае успешного выполнения и `NULL` в случае ошибки

Отображение объекта ShM

- Для отображения объекта ShM на адресное пространство процесса используется функция **MapViewOfFile**, принимающая:
 - **HANDLE hFileMappingObject** – дескриптор ShM, которая будет отображаться
 - **DWORD dwDesiredAccess** – флаги доступа, например:
 - **FILE_MAP_ALL_ACCESS** – полный доступ
 - **FILE_MAP_READ** – доступ на чтение
 - **FILE_MAP_WRITE** – доступ на запись
 - **FILE_MAP_EXECUTE** – доступ на выполнение
 - **DWORD dwFileOffsetHigh** – старшая часть сдвига относительно начала ShM (уст. в 0)
 - **DWORD dwFileOffsetLow** – младшая часть сдвига относительно начала ShM (уст. в 0)
 - **SIZE_T dwNumberOfBytesToMap** – количество отображаемых байт
- Функция возвращает указатель **LPVOID** на отображённый участок памяти (как **malloc**) в случае успеха и **NULL** в случае ошибки

Пример кода

- На следующем слайде представлен пример кода, в котором производятся описанные на предыдущих слайдах операции, и в полученное отображение записывается строка `Hello`
- Далее (на другом слайде) представлен код дочернего процесса, получающего эту строку и выводящего её на экран
- Обработка ошибок опущена для сокращения примера

Родительский процесс

```
HANDLE hMapping = CreateFileMappingA(INVALID_HANDLE_VALUE,  
                                     NULL,  
                                     PAGE_READWRITE,  
                                     0, 7, "pr7");  
  
char * buf = MapViewOfFile(hMapping,  
                            FILE_MAP_ALL_ACCESS,  
                            0, 0, 7);  
  
memcpy(buf, "Hello\n", 7);  
  
// Запуск дочернего процесса...
```

Дочерний процесс

```
HANDLE hMapping = OpenFileMappingA (FILE_MAP_ALL_ACCESS , FALSE, "pr7");  
  
char * buf = MapViewOfFile (hMapping, FILE_MAP_ALL_ACCESS , 0, 0, 7);  
  
DWORD written = 0;  
WriteFile (GetStdHandle (STD_OUTPUT_HANDLE), buf, 7, &written, NULL);
```

Удаление отображения

- Чтобы удалить отображение для данного процесса, используется функция `UnmapViewOfFile`, принимающая следующие аргумент:
 - `LPCVOID lpBaseAddress` – адрес удаляемого участка
- Удалённый участок больше недоступен для процесса
- Запись и чтение по адресам из такого участка приводит к неопределённому поведению
- Функция `UnmapViewOfFile` возвращает ненулевое значение в случае успеха и 0 в случае ошибки
- После удаления отображения записанные в него данные всё ещё сохраняются в объекте разделяемой памяти

Синхронизация

Средства синхронизации ShM

- В Windows задача синхронизации процессов упрощается благодаря тому, что мьютексы имеют уникальные имена
- Таким образом, для синхронизации доступа к общей памяти достаточно, чтобы:
 - Один процесс создал мьютекс с помощью `CreateMutex` и задал ему имя
 - Другие процессы открывали мьютекс с помощью `OpenMutex`
 - Другие процессы использовали мьютекс для синхронизации доступа к памяти, закрывая его с помощью `Wait`-функций и открывая с помощью `ReleaseMutex`
- Т.к. работа с мьютексами уже была подробно рассмотрена на следующем семинаре, дополнительные пояснения и примеры по данной теме приводиться не будут

Полезная литература

- Майкл Керриск – Linux API. Исчерпывающее руководство. – Книга, на основании которой сделана данная презентация. Для закрепления материала рекомендуются: глава 43, разделы 44.1-44.4 и глава 50. Для более углублённого ознакомления – главы 43-50.
- Джонсон М. Харт. Системное программирование в среде Windows. – Книга, на основании которой сделана часть про ShM в Windows. Для закрепления материала рекомендуется глава 5 раздел "Отображение файлов".
- [Документация](#) по функциям для работы с каналами в WinAPI